

DOSSIÊ TÉCNICO COMPLETO

MAPA DE ESPÉCIES AMEAÇADAS

Mapeamento Interativo Geoespacial da Biodiversidade na Região Sul do Brasil (PR, SC, RS): Arquitetura Híbrida de Software, Banco de Dados PostGIS, Ingestão SALVE/ICMBio e Fluxos de Integração.

SISTEMA / REPOSITÓRIO

TCC-MAPA-DE-ESPECIES

FOCO DA APRESENTAÇÃO

Apresentação Técnica & Defesa perante a Banca

ARQUITETURA E STACK

Node.js (BFF/Prisma) + Django REST + PostgreSQL/PostGIS + Leaflet.js

OBJETIVO DE ESTUDO

Otimizado para Indexação no NotebookLM e Simulação de Perguntas

Índice Temático de Estudo e Revisão

Este documento consolida a totalidade dos aspectos de arquitetura, código, banco de dados e engenharia de software do sistema **TCC-MAPA-DE-ESPECIES**. Ele foi projetado para leitura rápida antes da apresentação e para carregamento como fonte primária no **NotebookLM**, permitindo que você gere resumos em áudio, flashcards e simulações de perguntas e respostas.

Seção	Conteúdo Abordado	Objetivo Central na Apresentação
Seção 1	Visão Geral do Projeto e Resolução do Problema	Apresentar a dor real dos dados ambientais e o impacto social da solução.
Seção 2	Arquitetura Técnica e Comunicação entre Módulos	Defender com clareza o uso da arquitetura híbrida (Django + Node.js BFF).
Seção 3	Ferramentas, Tecnologias e Bibliotecas Utilizadas	Demonstrar domínio sobre cada biblioteca, SGBD e API externa do projeto.
Seção 4	Estrutura Técnica, Camadas e Fluxos de Dados	Explicar o caminho da informação do clique do mouse até o disco rígido.
Seção 5	Glossário Didático de Termos Técnicos para Leigos	Traduzir jargões difíceis (PostGIS, GeoJSON, BFF, GiST) em analogias simples.
Seção 6	Simulado da Banca: 10 Perguntas e Respostas	Treinar respostas prontas para os questionamentos mais duros da avaliação.
Seção 7	Conclusão Técnica e Horizontes de Expansão	Finalizar com postura madura sobre escalabilidade futura do sistema.

1. Visão Geral do Sistema e do Problema

1.1 O que é o Sistema?

O **Mapa de Espécies Ameaçadas** é uma plataforma web interativa desenvolvida para mapear, catalogar e representar visualmente a distribuição espacial de animais ameaçados de extinção nos três estados da Região Sul do Brasil (**Paraná, Santa Catarina e Rio Grande do Sul**). A plataforma integra, em uma única interface cartográfica, três entidades fundamentais da conservação ambiental: **Espécies Animais**, **Zonas de Preservação** (Unidades de Conservação e Parques Naturais) e **Organizações Não Governamentais (ONGs)** atuantes na causa ecológica.

Analogia Didática: Como explicar a ideia inicial para quem é leigo

Imagine uma biblioteca gigantesca do governo com milhares de caixas cheias de relatórios técnicos em papel e tabelas frias de números que ninguém fora da academia consegue entender. O nosso sistema atua como um **globo terrestre interativo e inteligente**: ele pega essas milhares de linhas de tabelas, descobre onde cada bicho vive de verdade e coloca um alfinete com foto, hábitos e alertas de perigo diretamente sobre o mapa da nossa região.

1.2 Qual problema real o sistema resolve?

A conservação da biodiversidade enfrenta desafios severos de **engenharia e comunicação da informação**:

- **Dispersão e Inacessibilidade de Dados Oficiais:** Os dados do *SALVE (Sistema de Avaliação do Risco de Extinção da Biodiversidade)* mantidos pelo ICMBio e Ministério do Meio Ambiente (MMA) costumam ser publicados em planilhas CSV com milhares de linhas e códigos técnicos de difícil interpretação pelo cidadão comum, escolas ou mesmo profissionais em campo.
- **Desconexão Espacial:** Saber que uma espécie como a *Onça-pintada (Panthera onca)* está classificada como "Vulnerável" ou "Criticamente em Perigo" é um fato abstrato. Visualizar em um mapa que ela sobrevive apenas em pequenas manchas isoladas de Mata Atlântica e constatar visualmente onde há áreas de proteção integral transforma a informação em consciência prática.
- **Escassez de Ferramentas Ágeis para Gestão:** Fiscais ambientais e pesquisadores muitas vezes não dispõem de sistemas leves onde seja possível clicar diretamente no mapa e registrar imediatamente um novo animal, foto ou entidade de proteção com coordenadas precisas.

1.3 Perfis de Usuário e Regras de Negócio do Projeto

Perfil	Tipo de Permissão	Principais Funcionalidades Disponíveis
Usuário Comum (Cidadão / Estudante)	Somente Leitura (Read-Only)	• Navegação livre pelo mapa interativo (zoom, pan, tela cheia). • Pesquisa instantânea por nome comum (ex: "Gato-maracajá") ou científico. • Filtros avançados por Bioma (Mata Atlântica, Pampa, Cerrado) e Nível de Risco. • Visualização de galeria de fotos, histórico e polígonos de distribuição. • Adição e gestão de animais na lista de Favoritos do usuário.
Administrador (Gestor / Pesquisador)	CRUD Completo (Create, Read, Update, Delete)	• Todas as funcionalidades do usuário comum. • Criação por Clique Direito (Contextmenu): captura automática da latitude e longitude exatas para criação instantânea de Animal, ONG ou Zona de Preservação. • Importador Automatizado do SALVE: upload de planilhas CSV com enriquecimento automático de imagens da Wikipédia e cálculo de coordenadas determinísticas. • Painel Administrativo nativo do Django para auditoria e controle de contas.

2. Arquitetura Técnica e Comunicação entre Componentes

2.1 A Decisão de Engenharia: Por que uma Arquitetura Híbrida?

Uma questão frequente em bancas avaliadoras é: *"Por que utilizar dois servidores backend (Django e Node.js) em vez de manter uma única tecnologia monolítica?"*. A resposta fundamenta-se no princípio de **Separação de Responsabilidades Especializadas**:

- **Django 6 (Python) — Núcleo de Governança, Regras de Negócio e Segurança:** O Django é mundialmente reconhecido por sua robustez corporativa. Ele possui um sistema nativo de autenticação altamente seguro (hashing PBKDF2), gestão de grupos de usuários, geração automática de migrações de banco de dados e um **Painel Administrativo completo pronto para uso**, evitando que a equipe tivesse que construir um painel de retaguarda do zero.
- **Node.js / Express 5 (JavaScript) — BFF (Backend for Frontend) de Alta Performance:** Aplicações de mapas geográficos operam com um perfil de uso muito específico: o usuário arrasta e amplia a tela continuamente, disparando rajadas de requisições de coordenadas espaciais. O Node.js possui um modelo de concorrência assíncrono e não-bloqueante (Event Loop) que entrega dados GeoJSON com consumo ínfimo de memória e latência de milissegundos. Além disso, gerencia transmissões em tempo real via *Server-Sent Events (SSE)* para importações em lote.

⚙️ O Padrão Arquitetural BFF (Backend for Frontend)

O padrão BFF determina que a API não deve ser genérica demais a ponto de forçar o navegador a fazer cálculos pesados. O servidor Node.js atua exatamente como esse facilitador: ele faz as consultas brutas no banco, calcula a agregação de fotos e formata tudo no padrão **GeoJSON FeatureCollection** que o Leaflet consome diretamente sem necessitar de nenhum processamento extra no cliente.

2.2 Diagrama Visual da Arquitetura do Sistema

Abaixo está a representação gráfica de como os componentes de frontend, os dois servidores backend e o banco de dados PostGIS se articulam:

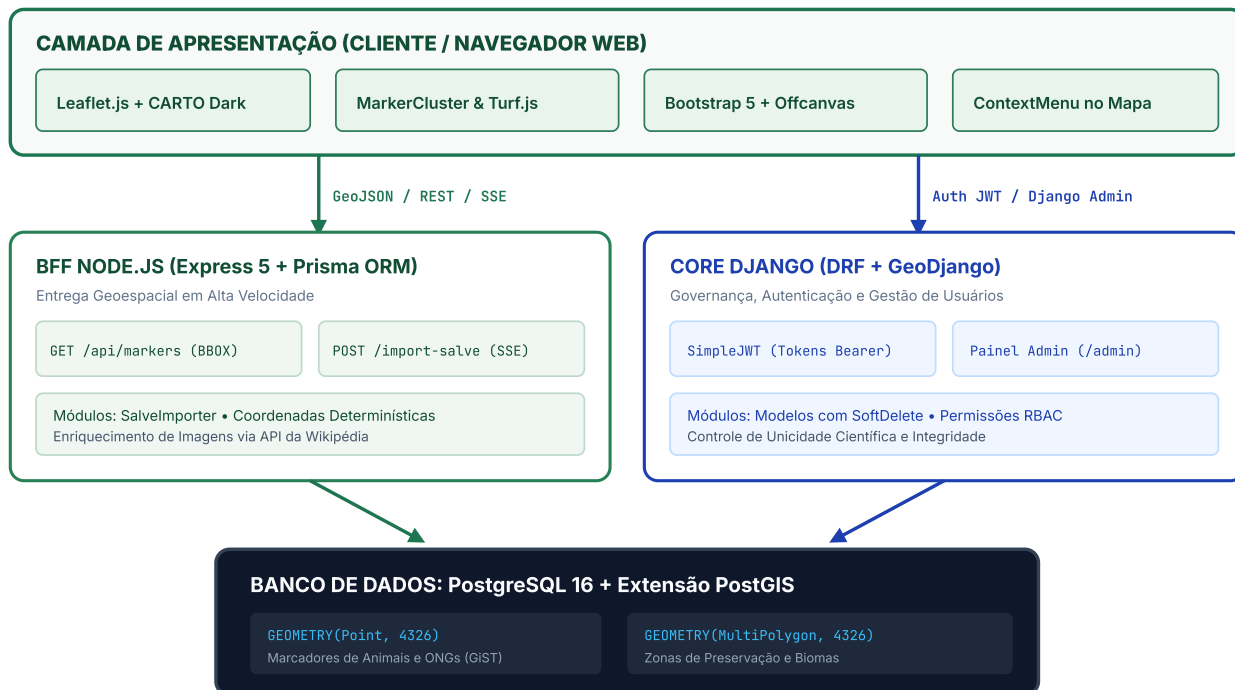


Figura 2.1 — Arquitetura Técnica Integrada do Sistema TCC-MAPA-DE-ESPECIES.

2.3 Protocolos e Modos de Comunicação entre Componentes

- Frontend ↔ BFF Node.js**: Quando o usuário movimenta o mapa, o Leaflet dispara uma requisição `GET /api/markers?bbox=minLng,minLat,maxLng,maxLat`. O Node.js executa a consulta espacial no PostGIS e responde com um objeto **GeoJSON FeatureCollection** contendo marcadores, fotos e biomas já agregados.
- Frontend ↔ Django API**: O formulário de login de administração envia credenciais para o endpoint do Django, que valida o usuário e devolve um par de **Tokens JWT (Access Token e Refresh Token)**. Requisições administrativas passam a carregar o cabeçalho `Authorization: Bearer <token>`.
- Compartilhamento Transparente do Banco de Dados**: O Django atua como o gestor oficial das tabelas (executando `python manage.py migrate`). O Prisma ORM no Node.js apenas faz a leitura estruturada do schema existente através de `npx prisma db pull`, garantindo que ambos operem sobre as mesmas tabelas sem qualquer conflito.

3. Ferramentas, Tecnologias e Bibliotecas Utilizadas

3.1 Detalhamento por Categoria Tecnológica

Abaixo detalha-se o papel de cada tecnologia adotada no ciclo de vida da aplicação:

Linguagens de Programação

- **JavaScript (Node.js no backend e ES6+ no navegador):** Utilizado tanto para a lógica reativa do mapa quanto para o servidor BFF. O modelo assíncrono do JavaScript permite processar grandes volumes de objetos JSON e requisições concorrentes sem engargalar a CPU.
- **Python 3.12+:** Empregado no backend corporativo (Django). Proporciona sintaxe limpa, tipagem moderna e suporte maduro a bibliotecas científicas e de banco de dados através do driver `psycopg2`.

Frameworks e Camada Servidora

- **Express.js (v5.2):** Framework minimalista para Node.js que gerencia as rotas rápidas do BFF, upload de arquivos através do middleware `multer` e entrega de mídias estáticas.
- **Django (v6.0) & Django REST Framework (v3.17):** Framework Python "baterias inclusas" responsável pela API REST corporativa, autenticação criptográfica, migrações de banco e administração do sistema.

Banco de Dados e Extensão Espacial

- **PostgreSQL 16 com PostGIS:** O SGBD relacional de código aberto mais avançado do mundo. A extensão PostGIS adiciona tipos primitivos de geometria (como pontos e polígonos) e funções que calculam interseções e distâncias reais sobre o esferoide terrestre.

Mapeamento Objeto-Relacional (ORMs)

- **Prisma ORM (v6):** ORM moderno para Node.js com segurança de tipos. Utilizado para consultas padrão e complementado com `$queryRawUnsafe` para comandos que requerem operadores nativos do PostGIS.
- **Django ORM / GeoDjango:** Mapeia classes Python diretamente em tabelas com suporte a campos espaciais (`PointField` e `MultiPolygonField`).

Bibliotecas do Frontend e Visualização Cartográfica

- **Leaflet.js (v1.9):** Mecanismo de renderização do mapa no navegador. Extremamente leve (cerca de 38 KB gzip), é o padrão de ouro para mapas interativos web.
- **Leaflet.markercluster:** Plugin que agrupa dezenas de marcadores geograficamente próximos em um único círculo numérico, prevenindo degradação de performance quando o mapa está com zoom aberto.
- **Turf.js:** Biblioteca de geoprocessamento no cliente (navegador). Efetua análises espaciais rápidas sem precisar fazer requisições ao servidor a todo instante.
- **Bootstrap 5.3 & FontAwesome 6.4:** Componentes de interface (gaveta lateral Offcanvas, modais, sistema de grid responsivo e ícones temáticos).
- **Pug 3 (Jade):** Motor de templates do Node.js que renderiza a estrutura básica do HTML com sintaxe limpa baseada em indentação.

3.2 Tabela Resumo do Stack Tecnológico

Tecnologia	Categoria	Função no Sistema	Justificativa de Escolha
Node.js + Express 5	BFF / SERVER	Entrega de dados GeoJSON, upload de imagens e streaming Server-Sent Events (SSE).	I/O assíncrono não-bloqueante; alta taxa de transferência para dados espaciais.
Django 6 + DRF	CORE / AUTH	Autenticação JWT, regras de negócio, exclusão lógica (Soft Delete) e painel administrativo.	Segurança robusta por padrão e interface administrativa pronta para auditoria.
PostgreSQL + PostGIS	SGBD ESPACIAL	Armazenamento de pontos (animais/ONGs) e polígonos (biomas/preservação) com índices GiST.	Padrão industrial para GIS; suporte a consultas espaciais ultrarrápidas na curvatura real da Terra.
Prisma ORM	ORM / QUERY	Execução de consultas tipadas no Node.js e gerenciamento de banco via Prisma Studio.	Alta produtividade e execução ágil de queries SQL espaciais combinadas.
Leaflet.js + MarkerCluster	GIS / FRONTEND	Renderização do mapa em tela cheia, azulejos de satélite/escuro e agrupamento de marcadores.	Leveza absoluta, código aberto, sem cobranças abusivas de API e alta velocidade de desenho em Canvas.
Turf.js 6	ANÁLISE CLIENTE	Cálculo geométrico no navegador (envelopes, distâncias e validação de coordenadas).	Execução local instantânea sem sobrecarregar a rede com chamadas desnecessárias ao servidor.
Bootstrap 5 + FontAwesome	UI / DESIGN	Interface de controle, Offcanvas lateral de edição/criação, modais e iconografia.	Design limpo, intuitivo e responsivo para smartphones, tablets e computadores.
Pug 3 (Jade)	TEMPLATE ENGINE	Geração do esqueleto HTML das páginas <code>index</code> , <code>animais</code> e <code>admin-login</code> .	Sintaxe expressiva e renderização eficiente integrada nativamente ao Express.
API Wikipédia / Wikimedia	API EXTERNA	Busca automatizada por nome científico de 1 a 3 fotos públicas de alta resolução por espécie.	Enriquecimento visual automático de bases governamentais brutas (SALVE/ICMBio).
SimpleJWT	SEGURANÇA	Geração e validação de tokens criptográficos Bearer para proteção de rotas administrativas.	Arquitetura stateless que permite autenticação unificada sem depender de sessões em disco.

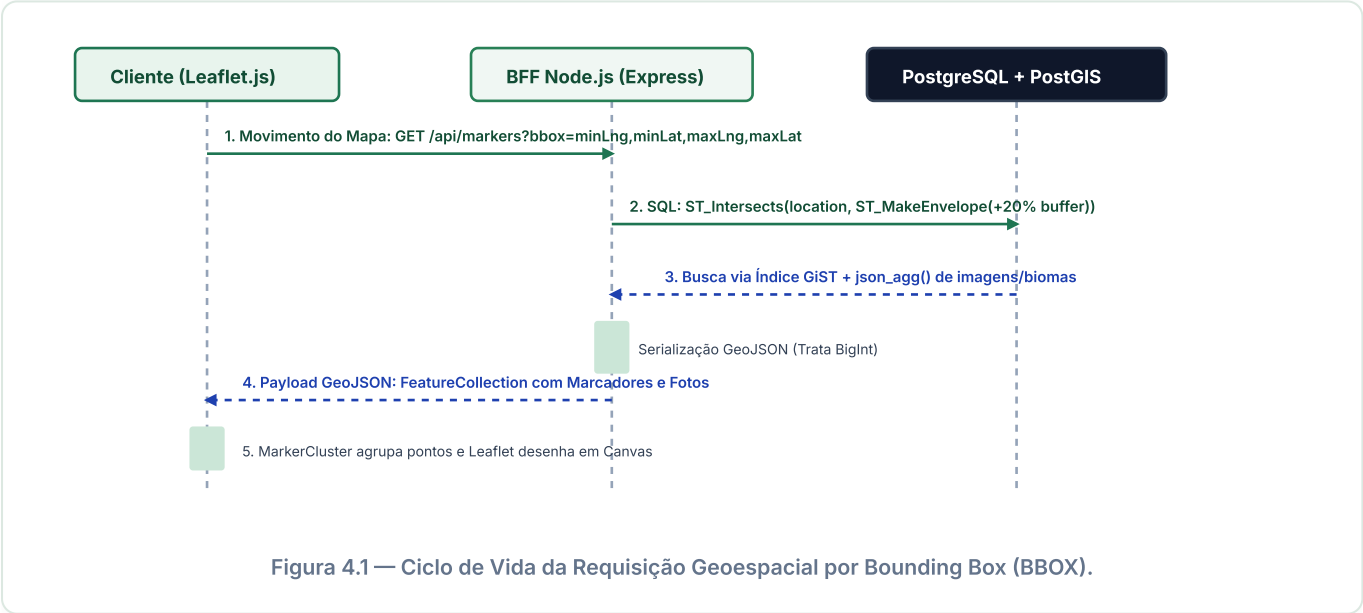
4. Estrutura Técnica, Camadas e Fluxos de Dados

4.1 Camada de Apresentação (Frontend no Navegador)

A experiência do usuário gira em torno do mapa geográfico, que foi desenhado para ser interativo e extremamente rápido:

- **Renderização em HTML5 Canvas (Aceleração por Hardware):** Ao configurar o Leaflet com `preferCanvas: true`, os marcadores de espécies não são criados como elementos HTML individuais no DOM (o que travaria o navegador com muitas entidades), mas sim desenhados diretamente em um elemento `<canvas>` acelerado pela GPU da máquina do usuário.
- **Filtro de Bounding Box (BBOX) com Buffer de 20%:** O navegador monitora a área visível da tela. Quando o usuário para de arrastar ou aproxima o zoom, o Leaflet dispara uma requisição contendo as coordenadas da tela. O backend calcula um **buffer de expansão de 20%** ao redor da área visível. Essa estratégia de pré-carregamento antecipa os próximos movimentos do usuário, garantindo que novos animais já estejam carregados na memória antes mesmo de entrarem na visão.
- **Menu de Contexto e Modo Administrador (Evento `contextmenu`):** O Leaflet escuta o clique com o botão direito do mouse no mapa. Se o usuário estiver autenticado com a permissão de administrador, o sistema captura a latitude e longitude exatas do clique e abre um menu flutuante. Ao selecionar *"Criar Animal"*, uma gaveta lateral (Offcanvas) se abre com as coordenadas já preenchidas no formulário, permitindo que o gestor adicione foto, nome científico e biomas em poucos segundos.

4.2 Diagrama de Sequência: Carregamento do Mapa (Fluxo BBOX)



4.3 O Mecanismo de Importação SALVE e Streaming em Tempo Real (SSE)

O arquivo `services/salveImporter.js` resolve um dos maiores desafios do projeto: processar bases governamentais brutas sem travar a interface.

- **Parser CSV Resiliente:** Trata variações de delimitadores (`;` , `,` ou tabulações), aspas escapadas e codificações tanto em UTF-8 quanto em Latin-1 (ISO-8859-1).
- **Algoritmo de Coordenadas Determinísticas (`getDistributedCoordinate`):** Quando a planilha oficial informa apenas que a espécie ocorre na "Mata Atlântica em Santa Catarina", sem ponto GPS exato, o sistema calcula uma coordenada consistente e espalhada dentro do polígono oficial do bioma naquele estado, impedindo que centenas de animais fiquem empilhados em um único ponto.
- **Server-Sent Events (SSE):** Em vez de um upload tradicional onde o usuário fica olhando para uma tela congelada, o endpoint `POST /api/v1/animais/import-salve?stream=true` abre um fluxo de eventos HTTP contínuo. A cada espécie cadastrada e enriquecida com fotos da Wikipédia, o servidor despacha um evento JSON com a porcentagem atual e o nome da espécie, movimentando a barra de progresso no navegador de forma suave.

4.4 Diagrama do Fluxo de Importação SALVE com Server-Sent Events

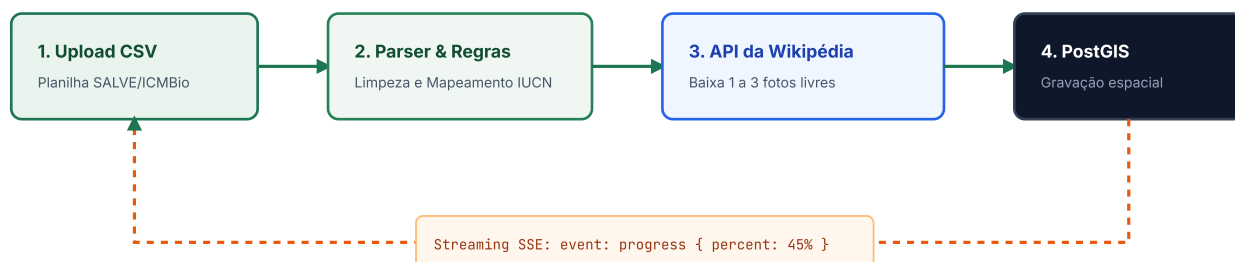


Figura 4.2 — Fluxo de Ingestão de Dados com Streaming de Eventos em Tempo Real (SSE).

4.5 Modelagem de Dados e Padrão Soft Delete

O modelo de dados implementado no PostgreSQL reúne informações taxonômicas, relações ecológicas e geometrias espaciais, contando com o padrão de **Soft Delete** para garantir que nenhuma exclusão acidental apague dados históricos de pesquisa:

Entidade / Tabela	Chave Primária / Tipo	Campos Chave e Atributos	Relacionamentos
api_animal	id (BigInt PK)	nome_comum , nome_cientifico (Unique), classe , dieta , deleted_at	FK para NivelExtincao, Microrregiao, User; N:N Biomias
api_marcaador	id (BigInt PK)	location (Point, 4326) [Índice GiST], icone , created_at	FK para api_animal (1:N)
api_animalimagem	id (BigInt PK)	imagem (VarChar 500) , legenda , ordem	FK para api_animal (1:N)
api_bioma	id (BigInt PK)	nome (VarChar 100) , caract , nivel_destr	N:N com api_animal e api_estado
api_ong	id (BigInt PK)	nome , site , location (Point, 4326) , deleted_at	Entidade de conservação georreferenciada
api_zonapreservacao	id (BigInt PK)	nome , categoria , area (MultiPolygon, 4326)	Polígonos de Unidades de Conservação
api_favorito	id (BigInt PK)	user_id , animal_id , created_at	Vínculo entre api_user e api_animal

5. Glossário Didático de Termos Técnicos para Leigos

Use este glossário quando precisar explicar conceitos avançados para membros da banca ou espectadores que não possuem formação em banco de dados ou geoprocessamento:

1. O que é GeoJSON?

Definição Acadêmica: É uma especificação aberta (IETF RFC 7946) que padroniza a representação de entidades geográficas e seus atributos utilizando a sintaxe do formato JSON.

Analogia para Leigos: É como a etiqueta de correio de um pacote digital. Em um arquivo de texto simples e leve, ele declara: *"Aqui dentro tem um Gato-Maracajá (nome e detalhes) e a casa dele fica exatamente nas coordenadas GPS: latitude -27.5, longitude -48.5"*. Qualquer aplicativo do mundo consegue ler essa etiqueta e cravar o alfinete no mapa na hora.

2. O que é PostGIS?

Definição Acadêmica: É uma extensão espacial para o SGBD PostgreSQL que adiciona suporte a objetos geométricos e geográficos e provê funções de cálculo espacial em conformidade com os padrões da OGC (Open Geospatial Consortium).

Analogia para Leigos: Um banco de dados normal é como uma planilha de escritório que só entende números e letras. O PostGIS é como colocar uma régua e um compasso geodésico dentro dessa planilha: ela passa a saber que a Terra é redonda, sabe calcular distâncias exatas em linha reta e consegue responder se uma floresta fica dentro ou fora de uma cidade em um piscar de olhos.

3. O que é Bounding Box (BBOX)?

Definição Acadêmica: É o menor retângulo espacial alinhado aos eixos cartesianos que delimita um conjunto de geometrias ou o campo de visão atual de um viewport de exibição.

Analogia para Leigos: Imagine olhar através de uma janela pequena para uma montanha. Você não vê o país inteiro, só vê o que cabe no retângulo de vidro daquela janela. O BBOX é o retângulo da janela do seu celular ou monitor. O sistema só carrega os animais que cabem naquela janela visível, economizando internet e bateria.

4. O que é Marker Clustering (Agrupamento de Marcadores)?

Definição Acadêmica: Algoritmo de condensação visual que agrega marcadores geograficamente próximos em um único ponto agregador para preservar a legibilidade e evitar saturação do mecanismo de renderização do navegador.

Analogia para Leigos: Quando você olha para uma floresta de dentro de um avião voando alto, você não consegue contar cada árvore ou cada macaco individualmente; você vê "um grupo de verde". Mas quando o avião pousa e você chega perto (zoom in), o grupo se desfaz e você vê cada árvore separadamente. Isso evita que o mapa vire um borrão vermelho ilegível.

5. O que é JWT (JSON Web Token)?

Definição Acadêmica: Padrão de autenticação stateless (RFC 7519) que trafega asserções de segurança assinadas digitalmente por chave criptográfica entre cliente e servidor.

Analogia para Leigos: É como uma **pulseira de acesso VIP inviolável** em um parque de diversões. Ao passar pela bilheteria (fazer login com senha), você recebe a pulseira. Em cada brinquedo que entrar depois, você não precisa assinar um livro ou mostrar seu documento novamente: basta exibir a pulseira para comprovar que tem permissão.

6. O que é Soft Delete (Exclusão Lógica)?

Definição Acadêmica: Padrão de persistência em que a remoção de um registro é sinalizada por um marcador temporal (campo `deleted_at`), preservando fisicamente a linha no disco e a integridade referencial histórica.

Analogia para Leigos: Em vez de rasgar uma folha de um livro e queimar no fogo (exclusão física), você coloca um carimbo vermelho escrito "arquivado" e guarda em uma pasta trancada. Para o público do mapa, o animal deixa de aparecer. Mas se um cientista precisar verificar dados históricos ou se alguém tiver clicado em excluir sem querer, a informação continua preservada no cofre do sistema.

6. Guia de Estudo: 10 Perguntas e Respostas da Banca de TCC

Revise este simulado técnico com atenção antes da apresentação. Ele cobre os questionamentos mais incisivos de arquitetura, banco e engenharia:

1. "Por que adotar uma arquitetura com dois servidores backend (Django e Node.js) em vez de manter uma única tecnologia?"

RESPOSTA IDEAL: Adotamos o padrão **BFF (Backend for Frontend)**. O **Django** é a nossa plataforma corporativa de governança: ele fornece o Painel Administrativo nativo, segurança pronta contra SQL Injection, CSRF e XSS, e gestão madura de permissões de usuários. O **Node.js** foi inserido como duto de alta vazão para o mapa: por ter um Event Loop assíncrono, ele entrega GeoJSON com latência mínima e processa streaming Server-Sent Events (SSE) sem travar threads de processamento. Ambos os serviços acessam o mesmo banco PostgreSQL de forma coordenada.

2. "Por que usar o PostGIS em vez de armazenar Latitude e Longitude como duas colunas Float convencionais?"

RESPOSTA IDEAL: Colunas numéricas (Float) tratam coordenadas como números planos bidimensionais e não possuem suporte a **Índices Espaciais (GiST)**. Para responder se um animal está dentro de uma reserva florestal irregular usando Float, o servidor precisaria carregar milhares de registros e calcular trigonometria esférica manualmente em código. Com o PostGIS, a localização é um tipo geométrico nativo (`Point`) indexado em árvore espacial (R-Tree), permitindo que o próprio banco responda a consultas espaciais como `ST_Intersects` em frações de milissegundos.

3. "Se o sistema tiver 50.000 animais cadastrados, como vocês garantem que o navegador não vai travar?"

RESPOSTA IDEAL: Implementamos três mecanismos de contenção de sobrecarga:

- **Bounding Box no Backend:** O servidor só envia os pontos da área visível na tela mais 20% de margem, nunca os 50.000 de uma vez.
- **Aceleração por Hardware via Canvas:** O Leaflet usa `preferCanvas: true` , desenhando marcadores na GPU em vez de sobrecarregar a árvore de elementos HTML (DOM).
- **Marker Clustering no Cliente:** Em zoom afastado, os pontos próximos são unificados em bolhas quantitativas, reduzindo centenas de nós a um único elemento visual leve.

4. "Como foi estruturada a segurança para evitar que usuários comuns alterem ou apaguem dados de espécies?"

RESPOSTA IDEAL: A segurança baseia-se em **Tokens JWT com Controle de Acesso Baseado em Papéis (RBAC)**. Usuários comuns têm apenas permissões de leitura (Read-Only). O administrador realiza login e recebe um token assinado criptograficamente com `role='admin'`. No frontend, as opções de clique direito e importação só são exibidas se o token for válido. No backend, todos os endpoints de escrita (POST, PATCH, DELETE) interceptam a requisição e barram qualquer chamada sem a assinatura digital do token administrativo.

5. "Como o sistema resolve a ausência de coordenadas exatas de GPS nas planilhas oficiais do SALVE/ICMBio?"

RESPOSTA IDEAL: Desenvolvemos a função `getDistributedCoordinate`. A planilha do governo frequentemente informa apenas o bioma e os estados onde a espécie habita (ex: "Mata Atlântica nos estados PR e SC"). O nosso algoritmo cruza esse registro com a malha poligonal oficial desses biomas e gera coordenadas geográficas determinísticas, plausíveis e espalhadas dentro da área natural no Sul do Brasil, impedindo que animais fiquem acumulados no oceano ou na coordenada zerada (0,0).

6. "Por que escolheram a biblioteca Leaflet.js em vez da API do Google Maps?"

RESPOSTA IDEAL: Por três pilares fundamentais:

- **Sustentabilidade Financeira:** A API do Google Maps cobra em dólar por cada requisição acima de uma cota básica, inviabilizando projetos públicos e acadêmicos. O Leaflet é 100% livre e de código aberto.
- **Independência de Fornecedor:** Com o Leaflet podemos alternar livremente entre mapas escuros da CARTO, fotos aéreas ou OpenStreetMap.
- **Eficiência de Código:** O Leaflet possui um núcleo de apenas 38 KB e excelente compatibilidade com plugins como MarkerCluster e Canvas rendering.

7. "Como funciona o fluxo de transmissão em tempo real durante a importação da planilha SALVE?"

RESPOSTA IDEAL: Adotamos a tecnologia **Server-Sent Events (SSE)**. Diferente de uma requisição HTTP comum que faz o navegador esperar a planilha inteira terminar em silêncio, o SSE estabelece um canal contínuo unidirecional. A cada espécie lida, gravada no PostGIS e enriquecida com fotos da Wikipédia, o Node.js envia uma linha com o evento `progress` contendo a porcentagem concluída e o nome da espécie. O JavaScript no navegador escuta esse evento e anima a barra de progresso em tempo real sem travar.

8. "Por que adotar o Prisma ORM no Node.js se o Django já tem seu próprio ORM nativo?"

RESPOSTA IDEAL: O Django ORM atua no ecossistema Python gerando as migrações e controlando o schema. O **Prisma ORM** foi adotado no Node.js por oferecer tipagem estática e segurança contra falhas de digitação em JavaScript. Em vez de escrever consultas SQL manuais para operações simples, usamos o Prisma Client com autocompletion. E para consultas espaciais avançadas com operadores do PostGIS, utilizamos o método `prisma.$queryRawUnsafe`, unindo segurança e desempenho.

9. "Como o sistema lida com a incompatibilidade de números inteiros de 64 bits (BigInt) entre o PostgreSQL e o JavaScript?"

RESPOSTA IDEAL: Chaves primárias de bancos de dados modernos usam `BigInt` para evitar esgotamento de identificadores. No entanto, a função nativa `JSON.stringify()` do JavaScript lança um erro crítico ao tentar serializar valores `BigInt` para envio pela rede. Criamos uma função de serialização personalizada que inspeciona o objeto e converte automaticamente qualquer `BigInt` para String antes da transmissão HTTP, garantindo integridade numérica sem gerar exceções no servidor.

10. "Se um administrador apagar uma espécie sem querer, como o sistema garante que o dado não seja destruído?"

RESPOSTA IDEAL: Aplicamos o padrão **Soft Delete (Exclusão Lógica)** no modelo `Animal`. O método `delete()` padrão foi sobrescrito para preencher o campo `deleted_at` com o horário atual, em vez de executar a instrução SQL destrutiva `DELETE`. O gerenciador de consultas padrão (Manager) filtra automaticamente as buscas para exibir apenas registros com `deleted_at` nulo. Assim, o registro desaparece imediatamente do mapa público, mas continua salvo no banco para auditoria ou restauração imediata via Painel Administrativo.

7. Conclusão Técnica e Horizontes de Expansão

7.1 Balanço do Trabalho de Engenharia

O desenvolvimento do **TCC-MAPA-DE-ESPECIES** consolidou a entrega de uma plataforma ambiental moderna, escalável e de alto impacto visual. O trabalho demonstrou que é plenamente viável transformar dados brutos e inacessíveis de órgãos de conservação em uma ferramenta visual interativa para a sociedade, ancorada em práticas de engenharia de software de padrão profissional:

- **Arquitetura Híbrida Harmoniosa:** Integração estável entre Python/Django e Node.js/Express operando concorrentemente sobre o mesmo banco de dados PostgreSQL.
- **Geoprocessamento Real com PostGIS:** Superação de abordagens amadoras de coordenadas Float, adotando suporte geoespacial nativo com índices GiST e projeção geodésica SRID 4326.
- **Automação de Ingestão e Enriquecimento:** Capacidade de ler planilhas oficiais do governo (SALVE/ICMBio) e enriquecê-las instantaneamente com fotografias de licença livre da Wikipédia e coordenadas determinísticas.
- **Desempenho Otimizado no Cliente:** Renderização com aceleração por GPU via Canvas, clustering inteligente e pré-carregamento periférico por Bounding Box com buffer de 20%.

7.2 Horizontes de Expansão Futura do Software

1. **Camada de Cache Distribuído com Redis:** Para suportar dezenas de milhares de usuários simultâneos em momentos de campanhas ecológicas, as respostas GeoJSON das cidades com maior densidade populacional (como Curitiba, Florianópolis e Porto Alegre) podem ser cacheadas em memória RAM no Redis.
2. **Sensoriamento Remoto e Satélites em Tempo Real:** Sobreposição de camadas de dados de satélite (como imagens do satélite Sentinel do programa Copernicus) para exibir alertas de desmatamento recente ou focos de calor/queimadas diretamente sobre as áreas onde as espécies vivem.
3. **Aplicativo Mobile Offline com PWA:** Permitir que agentes ambientais e biólogos realizem coletas de campo dentro de florestas sem sinal de internet, sincronizando os dados com o servidor central assim que o dispositivo recuperar conexão 4G/Wi-Fi.

Dica Final para a Apresentação na Banca

Mantenha a tranquilidade e fale com firmeza: destaque que **a engenharia de software foi utilizada como um instrumento prático de cidadania e preservação da fauna brasileira**. Enfatize que a arquitetura híbrida e a escolha do PostGIS foram decisões conscientes e fundamentadas para garantir que o sistema não seja apenas um protótipo visual, mas uma plataforma robusta pronta para produção. Muito sucesso na apresentação!